PROYECTO FINAL · DESARROLLO WEB

AEROLÍNEAS DEL ISTMO, S. A.

Sistema de Gestión de Reservas y Abordaje

Documentación del proceso de desarrollo
Domain Driven Design (DDD) · Arquitectura Hexagonal · Modelo C4
Implementación, pruebas y despliegue en servicios gratuitos.

ESTUDIANTE	INFORMACIÓN ACADÉMICA
Marcos Emanuel Méndez Ortega	Carné: 0905-23-5324
Ingeniería en Sistemas	Curso: Desarrollo Web
Sección "3"	Noviembre de 2026

DEMO PÚBLICA
https://mmarcos00.github.io/aerolineas-istmo/

REPOSITORIO
github.com/MMarcos00/aerolineas-istmo

Índice y guía de lectura

Este documento describe el desarrollo del prototipo académico desde el análisis del negocio hasta su publicación. En cada fase se explica **qué se hizo, por qué se hizo y cómo se resolvió**, además de incluir evidencias y fragmentos del código fuente.

FASE	CONTENIDO
01	Caso de estudio y alcance
02	DDD estratégico: glosario, subdominios y contextos
03	DDD táctico y reglas de negocio
04	Arquitectura Hexagonal: puertos y adaptadores
05	Modelo C4 y diagramas
06	Base de datos SQL y migraciones
07	Desarrollo de la interfaz y funciones
08	Pruebas funcionales y automatizadas
09	Publicación en Cloudflare y GitHub Pages
10	Evaluación, límites y conclusiones

ENLACES PARA EL CATEDRÁTICO

Aplicación: https://mmarcos00.github.io/aerolineas-istmo/
Código y archivos: https://github.com/MMarcos00/aerolineas-istmo
Diagramas editables: carpeta diagrams/

Criterio de evidencia: las capturas fueron proporcionadas durante el desarrollo. Algunas pertenecen a etapas anteriores y pueden mostrar cuatro vuelos o un mensaje temporal de API desconectada. Esas imágenes se identifican según su momento, sin presentarlas como el estado final de producción.

01 | Caso de estudio y alcance

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Analizamos las operaciones de ventas, reservas, equipaje, check-in y abordaje de una aerolínea regional.	El monolito descrito en el caso mezclaba pantallas, SQL y reglas; dificultaba cambios y aumentaba errores.	Dividimos el negocio en conceptos, políticas del dominio, servicios de aplicación y adaptadores tecnológicos.

Objetivo del sistema

Desarrollar un prototipo web que permita registrar clientes, consultar disponibilidad por ruta y fecha, crear reservas, confirmar cupos, emitir boletos, hacer check-in, registrar abordajes y notificar cambios de vuelo; todo con un núcleo de negocio desacoplado.

Tecnologías seleccionadas

CAPA / HERRAMIENTA	TECNOLOGÍA	MOTIVO
Interfaz	HTML + CSS + JavaScript	Ejecución simple en navegador y hosting estático.
Backend	Cloudflare Workers + TypeScript	API HTTP sin servidor propio y despliegue académico.
Base de datos	Cloudflare D1 (SQLite)	Persistencia SQL remota, migraciones y plan gratuito.
Código	WebStorm + Git + GitHub	Edición, versionado y evidencia de cambios.
Documentación	draw.io + PDF	Diagramas C4/DDD editables y documento entregable.

Alcance realmente construido

- **Implementado:** módulos de Pasajeros, Vuelos, Reservas, Aeropuerto y Notificaciones internas.
- **Datos de demostración:** 90 agentes y 40 pasajeros ficticios de carga inicial; los vuelos dependen de las migraciones aplicadas.
- **No implementado como integración real:** WhatsApp, pasarela bancaria, migración, quiosco, app móvil y autenticación de producción.



Figura 1. Flujo de negocio simplificado: desde un borrador hasta el abordaje. Elaboración a partir de los casos de uso implementados.

02 | DDD estratégico: lenguaje del negocio

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Definimos un vocabulario común y clasificamos subdominios y contextos delimitados.	Los equipos de ventas, aeropuerto y programación deben hablar de las mismas reglas sin confundir conceptos.	Registramos 32 términos en analysis/FASE_1_DDD.md y reutilizamos nombres como Pasajero, Reserva, Vuelo y Boleto en TypeScript.

Glosario resumido del lenguaje ubicuo

TÉRMINO	DEFINICIÓN DE NEGOCIO
Pasajero	Persona identificada que viajará.
Cliente titular	Responsable del pago o convenio.
Agencia de viaje	Cliente con reservas de grupo y crédito.
Corporativo	Empresa con convenio comercial.
Agente	Usuario de ventas que gestiona reservas.
Reserva	Conjunto de pasajeros y tramos con estado.
Reserva de grupo	Reserva de 10 o más viajeros autorizados.
Vuelo	Servicio con ruta, fecha, capacidad y código.
Tramo	Vuelo individual dentro de un itinerario.
Itinerario	Secuencia de uno o varios tramos.
Tarifa	Precio vigente por clase y vuelo.
Clase	Económica o ejecutiva.
Cotización	Cálculo de total, cargos y descuentos.
Boleto	Constancia con tarifa registrada al emitir.
Inventario	Cupos vendibles por clase y vuelo.
Sobreventa	Venta extra autorizada hasta 5% en económica.
Equipaje	Maleta facturada y su cargo.
Check-in	Validación de documentos y asiento.
Pase de abordar	Registro que habilita embarque.
Abordaje	Confirmación de ingreso al vuelo.
Manifiesto	Listado operativo de pasajeros de un vuelo.
Viajero frecuente	Categoría de fidelidad del pasajero.
Puerta	Lugar asignado para abordar.
Notificación	Aviso por cambio de vuelo.
Crédito disponible	Límite menos el crédito utilizado.
Penalidad	Recargo por cambio/cancelación.

TÉRMINO	DEFINICIÓN DE NEGOCIO
Reemisión	Boleto nuevo, con historial anterior.
Reembolso simulado	Importe teórico tras penalidad.
Documento de viaje	Identificación y vencimiento para viajar.
Estado de reserva	Borrador, confirmada, emitida, etc.
Asiento	Lugar físico asignado al check-in.
Pago simulado	Registro financiero sin cobro real.

Glosario completo y explicación de términos: **analysis/FASE_1_DDD.md**.

02 | Subdominios y Bounded Contexts

Clasificación estratégica

TIPO	SUBDOMINIO	JUSTIFICACIÓN
Core	Reservas y Tarifas	Contiene grupos, precios, descuentos, crédito, cupos y emisión.
Soporte	Vuelos; Inventario de asientos; Pasajeros; Operaciones de aeropuerto	Ofrecen información y funciones necesarias para ejecutar las ventas y el viaje.
Genérico	Pagos; Notificaciones; Autenticación	Podrían sustituirse por productos externos; aquí se usa una simulación o diseño futuro.

Contextos delimitados

Se definieron **Vuelos, Reservas y Tarifas, Inventario, Pasajeros y Clientes, Operaciones de aeropuerto, Pagos y Notificaciones**. Cada uno maneja una perspectiva de negocio. Por ejemplo, “Pasajero” en Reservas es parte del itinerario; en Aeropuerto es la persona que se presenta para check-in y abordaje.

Mapa de contextos (Context Map)

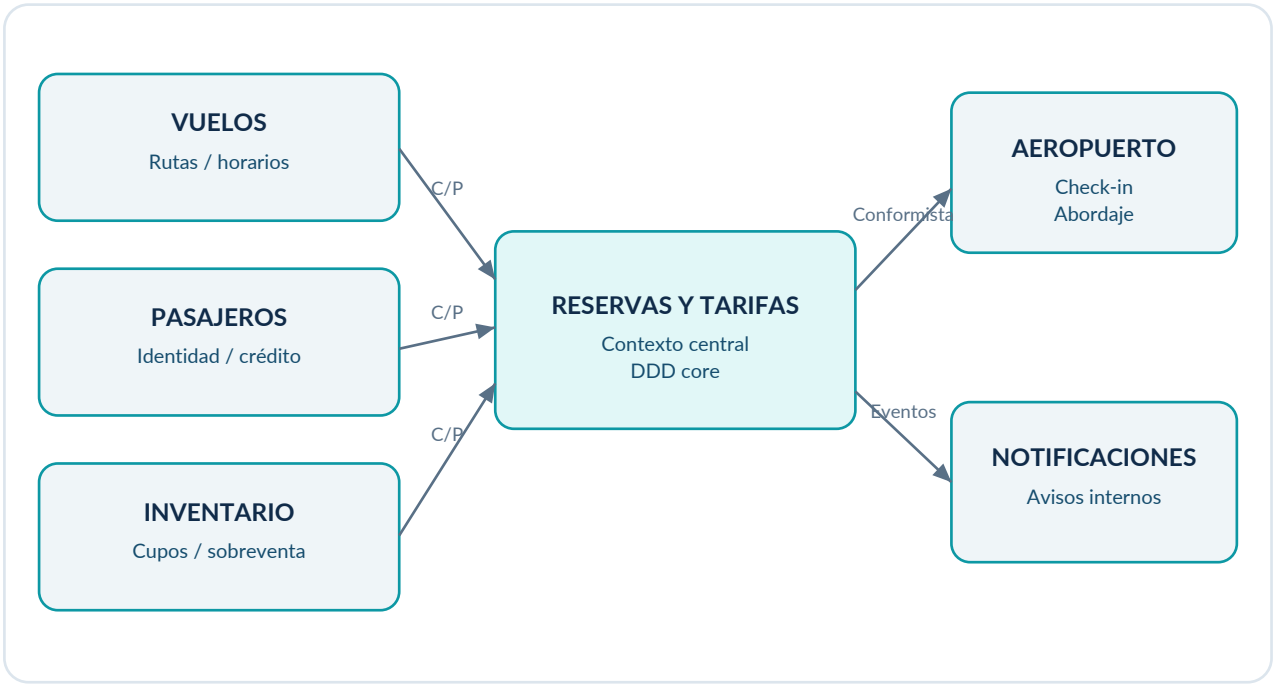


Figura 2. Mapa simplificado de relaciones. C/P = Cliente/Proveedor; el mapa original ampliado también documenta relaciones Conformista, Kernel compartido y ACL para integración futura.

DECISIÓN ARQUITECTÓNICA

Los contextos son separaciones **lógicas** de este prototipo: no se desplegaron como microservicios. La Capa Anticorrupción (ACL) frente a pagos y migración es parte del diseño previsto, no una integración externa activa.

03 | DDD táctico y reglas de negocio

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Identificamos entidades, agregados y políticas: Reserva, Pasajero, Vuelo, Boleto y PoliticaDeReservas.	Las reglas RN-01 a RN-08 deben vivir en el dominio, no dentro de formularios o consultas SQL.	Modelamos tipos en api/src/domain/model.ts y validaciones en api/src/domain/policies.ts, invocadas desde los casos de uso.

Principales invariantes del dominio

REGLA	COMPORTAMIENTO
RN-01	1–9 pasajeros individuales; desde 10 solo agencias/corporativos.
RN-02	Crédito disponible para convenio; pago individual antes de emitir.
RN-03	Capacidad por clase; sobreventa de hasta 5% solo económica autorizada.
RN-04	Más de 10 pasajeros: 5%; más de 30: 10%.
RN-05	Cancelar solo en estados permitidos y con 3 horas de anticipación.
RN-06	Documento con al menos seis meses vigentes para vuelo internacional.
RN-07	Sin edición directa del boleto emitido; cambio/cancelación con recargo simulado.
RN-08	Tarifa del boleto histórica e independiente de futuras modificaciones.

Ejemplo real: descuento y sobreventa en dominio

api/src/domain/policies.ts — política RN-04

```
static porcentajeDescuento(pasajeros: number): number { // RN-04
    return pasajeros > 30 ? 0.10 : pasajeros > 10 ? 0.05 : 0;
  }
```

Máquina de estados de una reserva

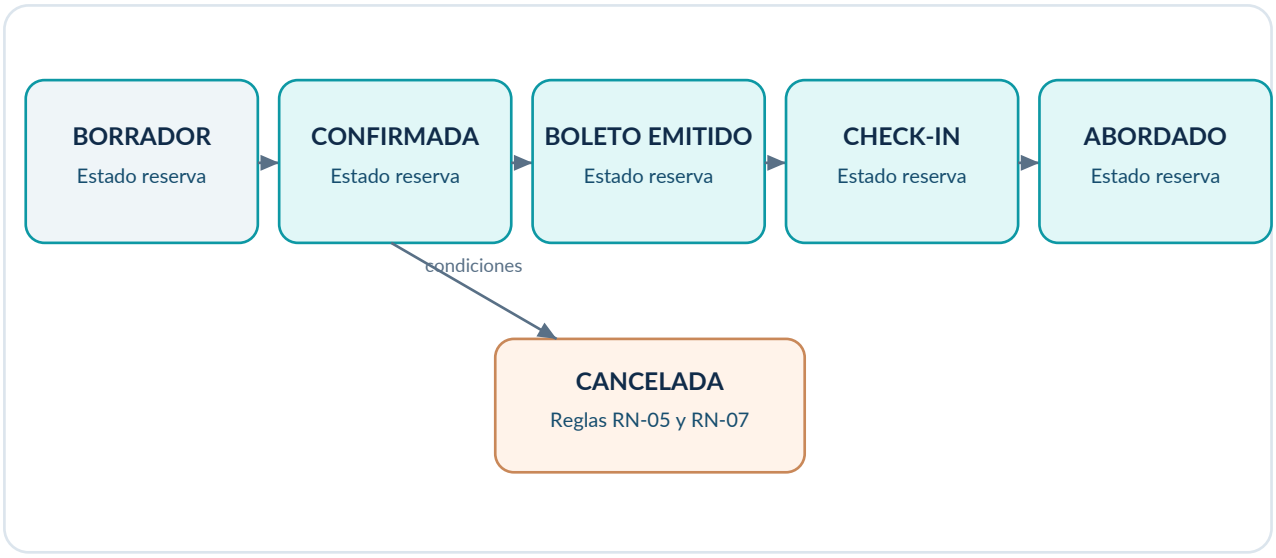


Figura 3. Transiciones principales. La cancelación está sujeta al estado y al horario de salida; los cambios de boleto siguen un caso de uso separado.

04 | Arquitectura Hexagonal

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Separamos adaptador de entrada HTTP, servicio de aplicación, política de dominio, puerto e implementación SQL.	Una nueva app móvil, interfaz de agencia o proveedor de datos no debe exigir reescribir todas las reglas de reserva.	El Worker invoca AerolineaService; este depende de PuertoRepositorioAerolinea; D1Repositorio implementa el puerto.

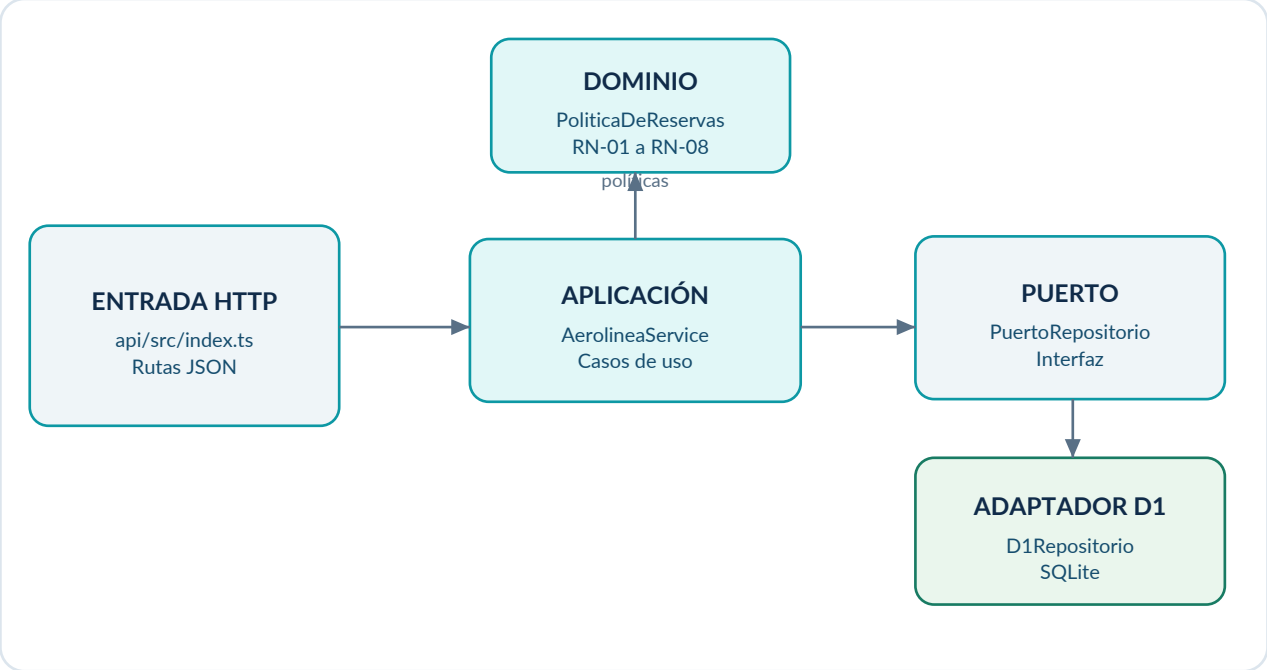


Figura 4. Puertos y adaptadores. Las flechas indican coordinación de casos de uso y acceso a persistencia; las políticas no conocen Cloudflare ni SQL.

Ubicación concreta de las capas

CAPA	ARCHIVO	RESPONSABILIDAD
Entrada	api/src/index.ts	Rutas HTTP, JSON y CORS.
Aplicación	api/src/application/AerolineaService.ts	Confirma, paga, emite, cambia, cancela y consulta.
Dominio	api/src/domain/model.ts / policies.ts	Entidades y políticas RN-01 a RN-08.
Puerto	api/src/application/ports.ts	Contrato de persistencia independiente.
Salida	api/src/infrastructure/D1Repositorio.ts	Traduce contratos a consultas D1 SQLite.

Fragmento del contrato de persistencia

api/src/application/ports.ts — fragmento simplificado del contrato real

```
export interface PuertoRepositorioAerolinea {
  pasajeros(): Promise<Pasajero[]>;
  vuelo(id: string): Promise<Vuelo | null>;
  confirmar(reserva: Reserva, total: number,
            equipajes: number, titular: Pasajero): Promise<void>;
  emitir(reservaId: string, boletos: Boleto[]): Promise<void>;
}
```

05 | Modelo C4: cuatro niveles

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Documentamos el contexto, contenedores, componentes y clases de la solución.	Distintos públicos necesitan diferentes niveles de detalle para comprender el sistema.	Creamos el archivo draw.io Arquitectura_Aerolineas_Istmo.drawio con cuatro C4, un Context Map y un esquema SQL.

Nivel 1 — Contexto del sistema

Usuarios: agentes de ventas y personal de aeropuerto. Sistema central: gestión de reservas y abordaje. Sistemas de pago y migración se representan como dependencias futuras, con ACL propuesta.

Nivel 2 — Contenedores

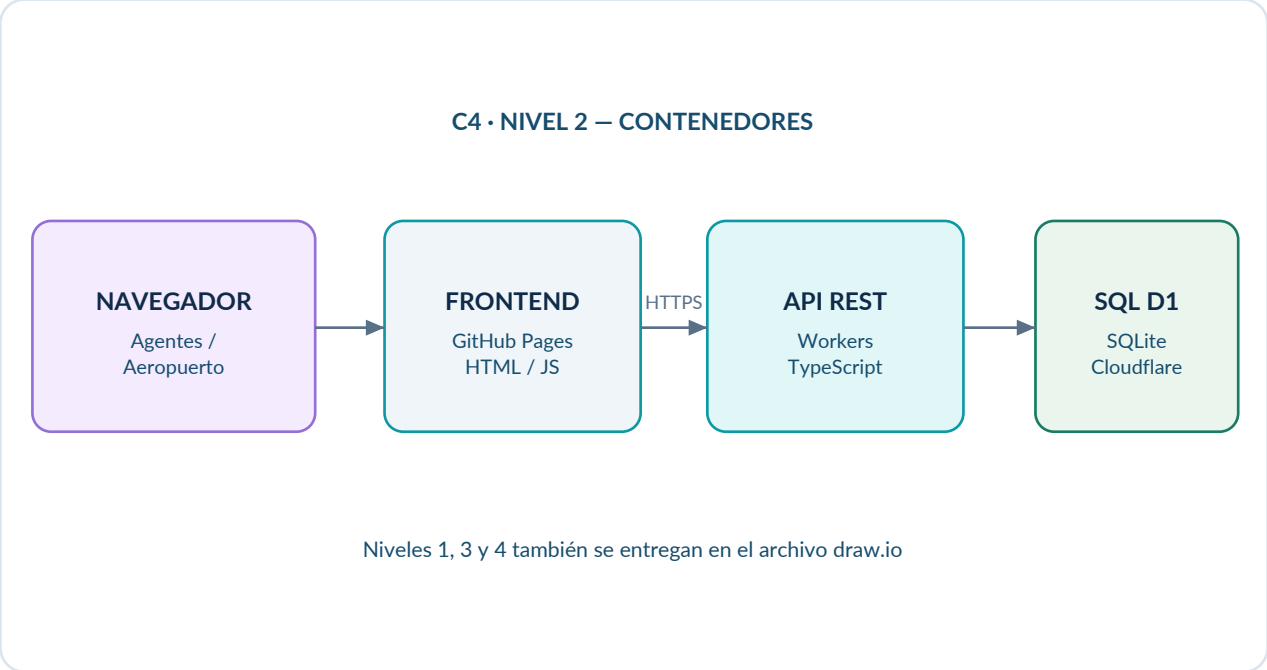


Figura 5. Contenedores desplegados: navegador → GitHub Pages → API Cloudflare Workers → base Cloudflare D1.

Nivel 3 — Componentes

Controlador HTTP (**index.ts**); casos de uso (**AerolineaService**); reglas (**PoliticaDeReservas**); interfaz (**PuertoRepositorioAerolinea**); adaptador SQL (**D1Repositorio**).

Nivel 4 — Código

Las clases y contratos muestran qué depende de qué. La dirección de dependencia se mantiene orientada al dominio y al puerto, para facilitar pruebas sin acceso a Internet.

ARCHIVOS EDITABLES PARA EL DOCENTE
Abrir diagramas C4 y modelo de datos (.drawio) Abrir diagrama del cambio de boleto RN-07

06 | Base de datos SQL y migraciones

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Diseñamos tablas de pasajeros, vuelos, reservas, boletos, pagos, check-in y cambios con penalidad.	La persistencia era necesaria para que distintos usuarios consultaran la misma información desde el sitio público.	Usamos Cloudflare D1 con scripts incrementales de migración 0001 y 0002, manteniendo modelos y políticas en TypeScript.

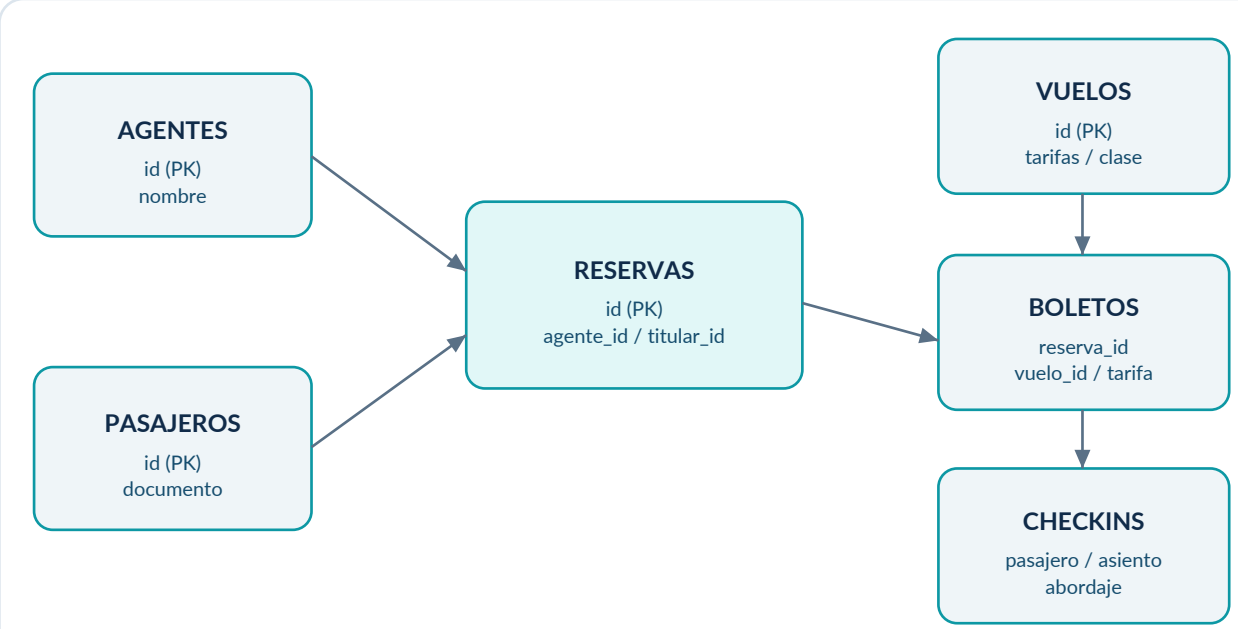


Figura 6. Modelo relacional simplificado de las entidades principales. El diagrama completo editable está en diagrams/.

Migraciones ejecutadas

MIGRACIÓN	CAMBIOS	RESULTADO OBSERVADO
0001_inicial.sql	Estructura principal e inserción de datos ficticios.	21 instrucciones ejecutadas; 90 agentes y 40 pasajeros en pruebas iniciales.
0002_cambios_con_penalidad.sql	Columnas de penalidad, tablas de cambios/cancelaciones y vuelos alternos.	9 instrucciones ejecutadas en base D1 remota, reportadas sin error.

Fragmento de SQL de la ampliación

api/migrations/0002_cambios_con_penalidad.sql — extracto

```
ALTER TABLE boletos
  ADD COLUMN anulado INTEGER NOT NULL DEFAULT 0;
CREATE TABLE IF NOT EXISTS cambios_boleto (
  id TEXT PRIMARY KEY,
  reserva_id TEXT NOT NULL REFERENCES reservas(id),
  vuelo_anterior_id TEXT NOT NULL REFERENCES vuelos(id),
  vuelo_nuevo_id TEXT NOT NULL REFERENCES vuelos(id),
  penalidad INTEGER NOT NULL,
  diferencia_tarifaria INTEGER NOT NULL,
  recargo_total INTEGER NOT NULL,
  fecha TEXT NOT NULL
);
```

La base **local** y la base **remota** son distintas. Se comprobó la remota con `wrangler d1 execute ... --remote`; no debe asumirse sincronización automática.

07 | Interfaz web y módulos

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Construimos un panel navegable con Pasajeros, Vuelos, Reservas, Aeropuerto y Notificaciones.	Los agentes y el personal de aeropuerto necesitan un flujo sencillo que muestre la información de la base y permita acciones.	Diseñamos HTML/CSS responsive; docs/app.js coordina formularios; docs/api-client.js consume la API y docs/config.js fija la URL.

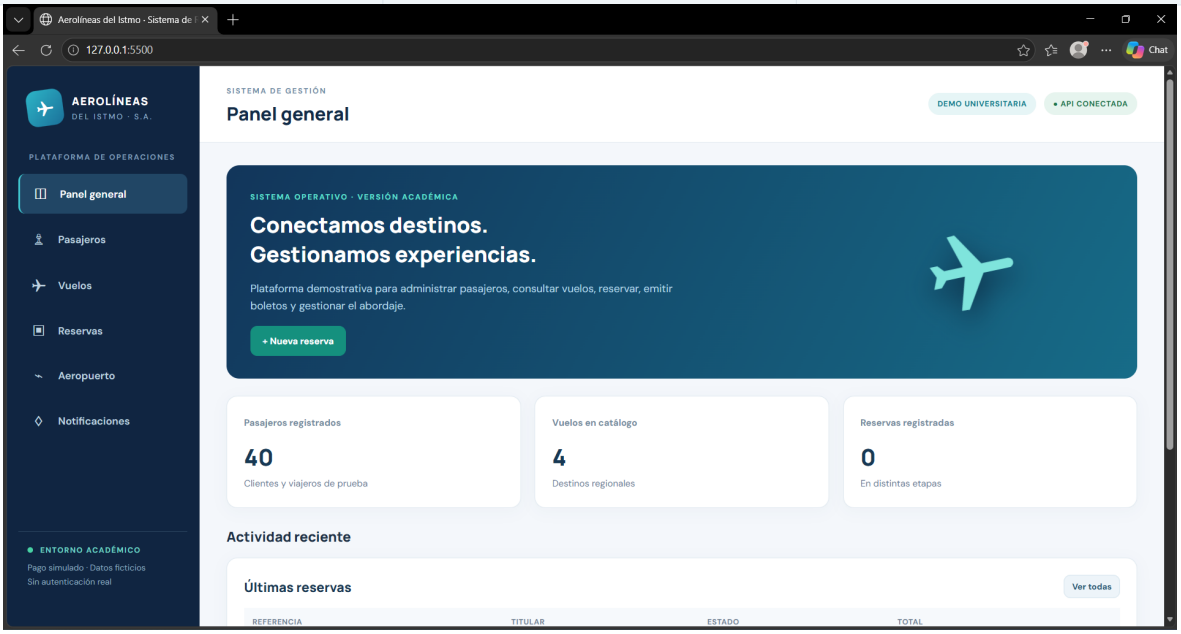


Figura 7. Panel general, durante las primeras pruebas locales. Se observan 40 pasajeros, 4 vuelos y conexión a la API en ese momento.

Acciones por módulo

MÓDULO	FUNCIONES PRINCIPALES
Panel general	Indicadores y últimas reservas.
Pasajeros	Alta y consulta por tipo de cliente, documento, fidelidad y crédito.
Vuelos	Filtros origen/destino/fecha, clase, tarifa, cupos y cambios operativos.
Reservas	Borrador, edición de tramos y viajeros, cotización, confirmación, pago simulado, emisión y RN-07.
Aeropuerto	Check-in, asiento, equipaje, abordaje y manifiesto.
Notificaciones	Buzón interno de cambios de horario, puerta y cancelación.

Conexión con la API publicada

docs/config.js

```
window.API_URL = 'https://aerolineas-istmo-api.mendezmarcosgt.workers.dev';
```

07 | Búsqueda de vuelos y resolución de errores

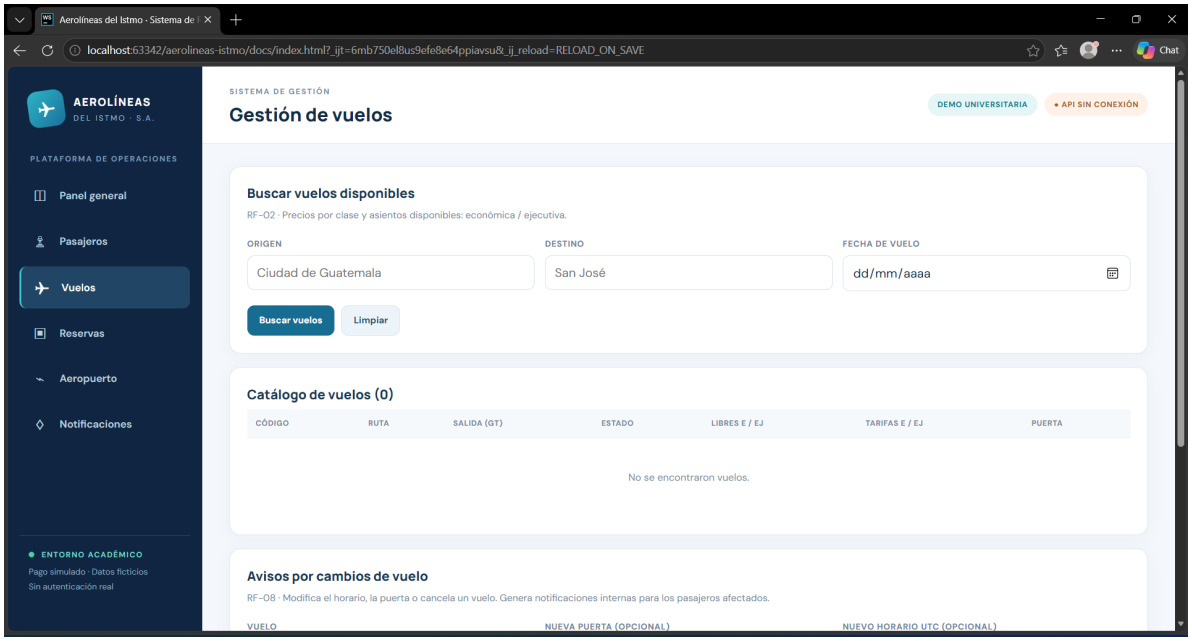


Figura 8. Captura de una prueba en el navegador de WebStorm donde apareció "API sin conexión". Se usa como evidencia de incidencia, no como prueba de éxito.

Problema detectado. En una etapa de las pruebas, el frontend abierto por el servidor interno de WebStorm no pudo comunicarse con la API. Esto explica la tabla de vuelos vacía en esa captura.

Solución aplicada. Se configuró la URL absoluta del Worker en `docs/config.js`, se habilitó la dirección pública `workers.dev` y se probó el sitio desde su URL de GitHub Pages. La presencia de los datos en las capturas locales anteriores no demuestra por sí sola el estado de la web pública actual.

PRUEBA FUNCIONAL PROPUESTA

Abrir la página pública, verificar el estado de la API, entrar a Vuelos y comprobar que muestre el catálogo remoto. Después crear una reserva de prueba y confirmar que también aparece en otra sesión/dispositivo.

08 | Pruebas automáticas y verificación

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Definimos pruebas unitarias del dominio y de integración de flujos HTTP con SQLite de prueba.	Las reglas críticas deben poder verificarse sin depender del navegador o de la base remota.	Compilamos TypeScript y ejecutamos scripts de Node; se reportaron 9 pruebas de dominio y 12 de integración exitosas.

COMANDO	PROPÓSITO	RESULTADO REGISTRADO
npm run types	Comprobar TypeScript sin emitir archivos.	Sin errores.
npm run test	RN-01 a RN-07 y penalidades.	9 aprobadas, 0 fallidas.
npm run test:integration	Flujos HTTP, cupos, pago, emisión, check-in, notificaciones, manifiesto.	12 aprobadas, 0 fallidas.

Registro de la terminal compartido durante el proyecto

Resumen textual de ejecución aportado en la conversación; no es una captura de consola

```
> npm run types
> tsc --noEmit

> npm run test
[dominio] tests 9 | pass 9 | fail 0

> npm run test:integration
[integracion] tests 12 | pass 12 | fail 0
```

Casos críticos comprobados

- RN-02: un pasajero individual no puede emitir boleto sin registrar pago simulado.
- RN-03/RF-05: al confirmar una reserva se descuenta el cupo en secuencia.
- RN-06: check-in internacional rechazado por documento con vencimiento insuficiente.
- RN-07: cambio de boleto con penalidad y preservación del boleto histórico; cancelación con recargo.
- RN-08: un cambio posterior de tarifa no modifica el valor histórico almacenado.
- RF-08: cambio de puerta genera una notificación interna para pasajeros vinculados.

Alcance de la prueba: la integración usa SQLite de prueba; no equivale a una prueba de concurrencia real ni prueba funcional automática en GitHub Pages.

09 | Despliegue en la nube

QUÉ HICIMOS	POR QUÉ	CÓMO LO RESOLVIMOS
Creamos D1 remota, publicamos el Worker y subimos el repositorio público a GitHub Pages.	El docente debía acceder desde su propio dispositivo sin instalar el proyecto ni compartir la base local.	Usamos Wrangler OAuth, migraciones remotas, wrangler deploy, git push y Pages desde main /docs.

Paso 1. Base D1 y publicación del backend

Comandos ejecutados en WebStorm / PowerShell

```
cd api
npm install
npx wrangler login
npx wrangler d1 create aerolineas-istmo-db
npm run db:remota
npx wrangler deploy
```

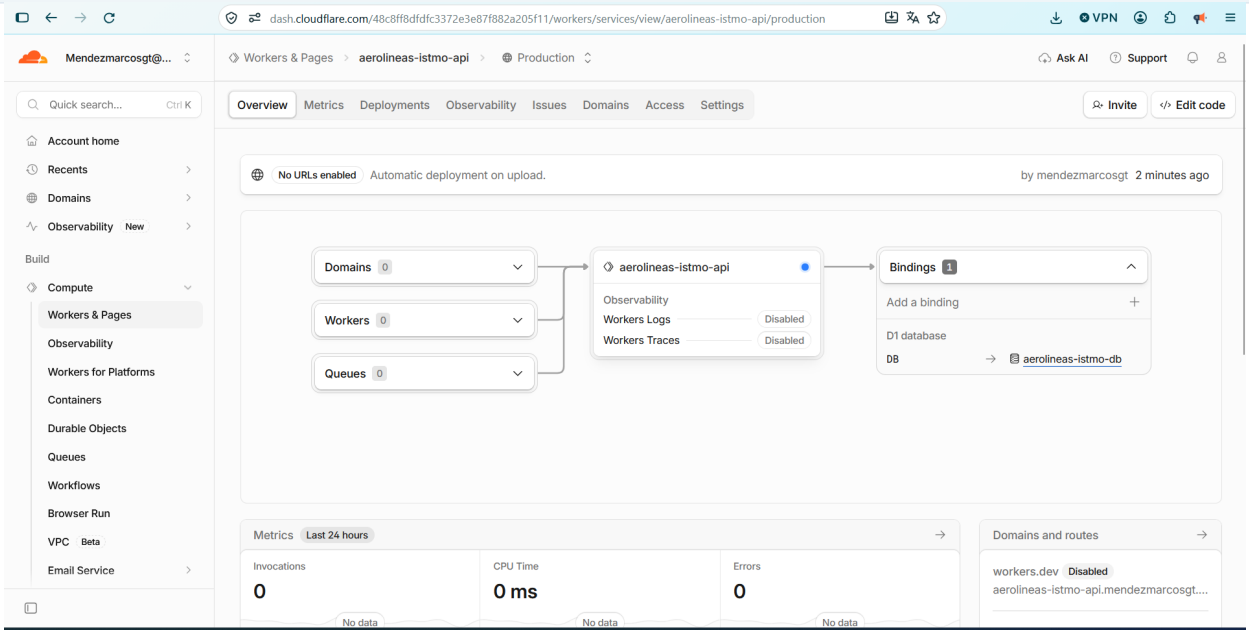


Figura 9. Vista de Cloudflare Workers con el Worker y el enlace de base de datos D1. Captura de una etapa de configuración anterior.

El despliegue final informado en terminal fue exitoso y generó la versión `d9f799c3-....` Los nombres y el identificador interno de D1 se conservan en la configuración del repositorio.

09 | GitHub Pages y acceso del docente

Paso 2. Repositorio y publicación estática

Se inicializó Git, se configuró la rama `main`, se subieron los archivos y se habilitó GitHub Pages desde la carpeta `/docs`. La actualización RN-07 quedó registrada en el commit `bcacc37`.

Comandos y configuración del despliegue en GitHub

```
git init
git branch -M main
git remote add origin https://github.com/MMarcos00/aerolineas-istmo.git
git add .
git commit -m "proyecto aerolineas del istmo"
git push -u origin main
# Pages: Settings > Pages > main > /docs
```

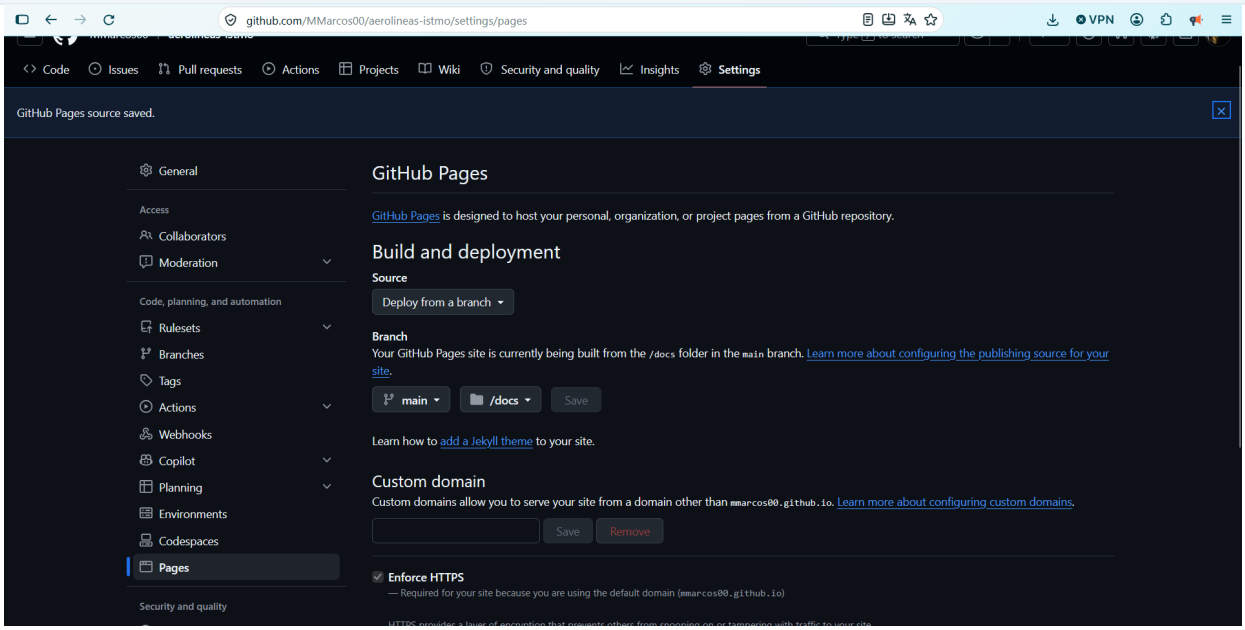


Figura 10. GitHub Pages configurado en “Deploy from a branch”, rama main y carpeta /docs.

Enlaces finales de la entrega

APLICACIÓN Y CÓDIGO

Página para evaluación: https://mmarcos00.github.io/aerolineas-istmo/
Repositorio: https://github.com/MMarcos00/aerolineas-istmo
Endpoint de salud: https://aerolineas-istmo-api.mendezmarcosgt.workers.dev/health

10 | Matriz de cumplimiento y cierre

CRITERIO	EVIDENCIA	VALORACIÓN
DDD estratégico	Glosario, subdominios, contextos y Context Map.	Documentado
Hexagonal	Dominio + servicios + puertos + adaptador SQL.	Implementado
C4	Cuatro niveles, plus SQL y mapa de contextos.	Documentado
RF-01 a RF-04	Pasajeros, búsqueda, reservas, cotización.	Demo funcional
RF-05	Cupos descontados al confirmar.	Sin atomicidad concurrente
RF-06 y RF-07	Estados e historial por filtros.	Demo funcional
RF-08	Mensajes internos por cambios de vuelo.	Sin envíos externos
RN-01 a RN-08	Políticas de dominio, controles, tarifa histórica, RN-07 ampliada.	Probado en flujos locales
Despliegue	Worker + D1 + GitHub Pages.	Publicado según registros

Limitaciones conocidas que deben informarse

- Sin autenticación ni permisos por rol: la demo pública solo debe utilizar datos ficticios y puede ser modificada por visitantes.
- Pagos, penalidades y reembolsos simulados; sin dinero ni integraciones bancarias reales.
- Sin garantía de confirmación de cupos atómica ante múltiples agentes al mismo tiempo.
- Notificaciones solo dentro de la aplicación; no se enviaron mensajes a correo, SMS o WhatsApp.
- Las entidades de clientes comerciales están simplificadas; tampoco existe una app móvil o quiosco desarrollados.
- Las capturas documentan distintas etapas de prueba; verificar siempre los datos actuales en el enlace público.

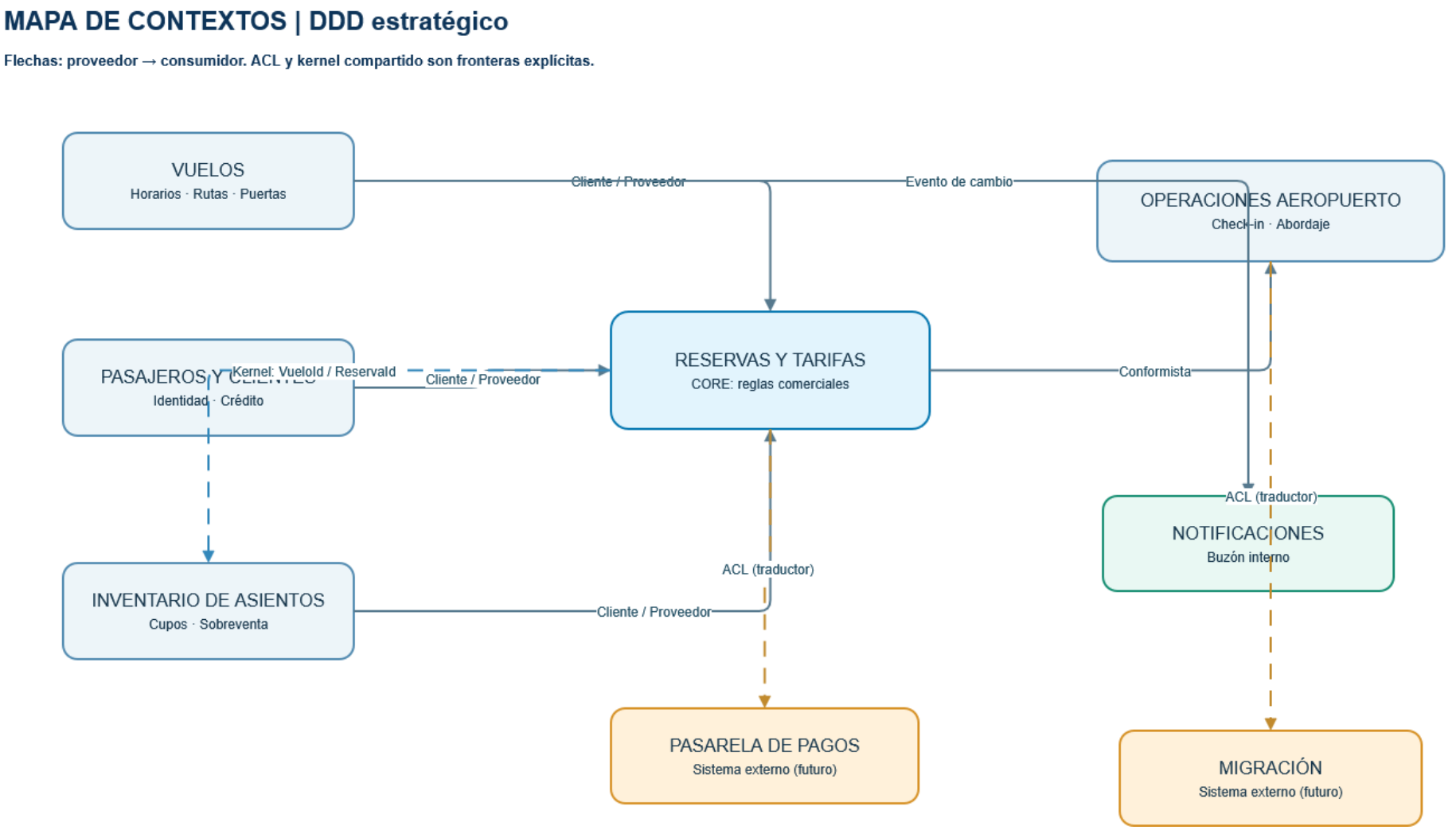
Conclusiones del desarrollo

El prototipo demuestra que es posible separar las reglas del negocio de la interfaz y de SQL para facilitar mantenimiento y pruebas. DDD ayudó a definir el vocabulario; la arquitectura hexagonal aisló los detalles de Cloudflare y el modelo C4 permitió explicar el diseño a distintas audiencias. La base remota y el despliegue público permiten presentar el sistema sin depender de la computadora del estudiante.

El producto se presenta como **prototipo académico verificable**, no como plataforma comercial de venta real de boletos. Esto permite demostrar las ocho reglas y los flujos principales sin ocultar riesgos técnicos pendientes.

MAPA DE CONTEXTOS · DDD ESTRATÉGICO

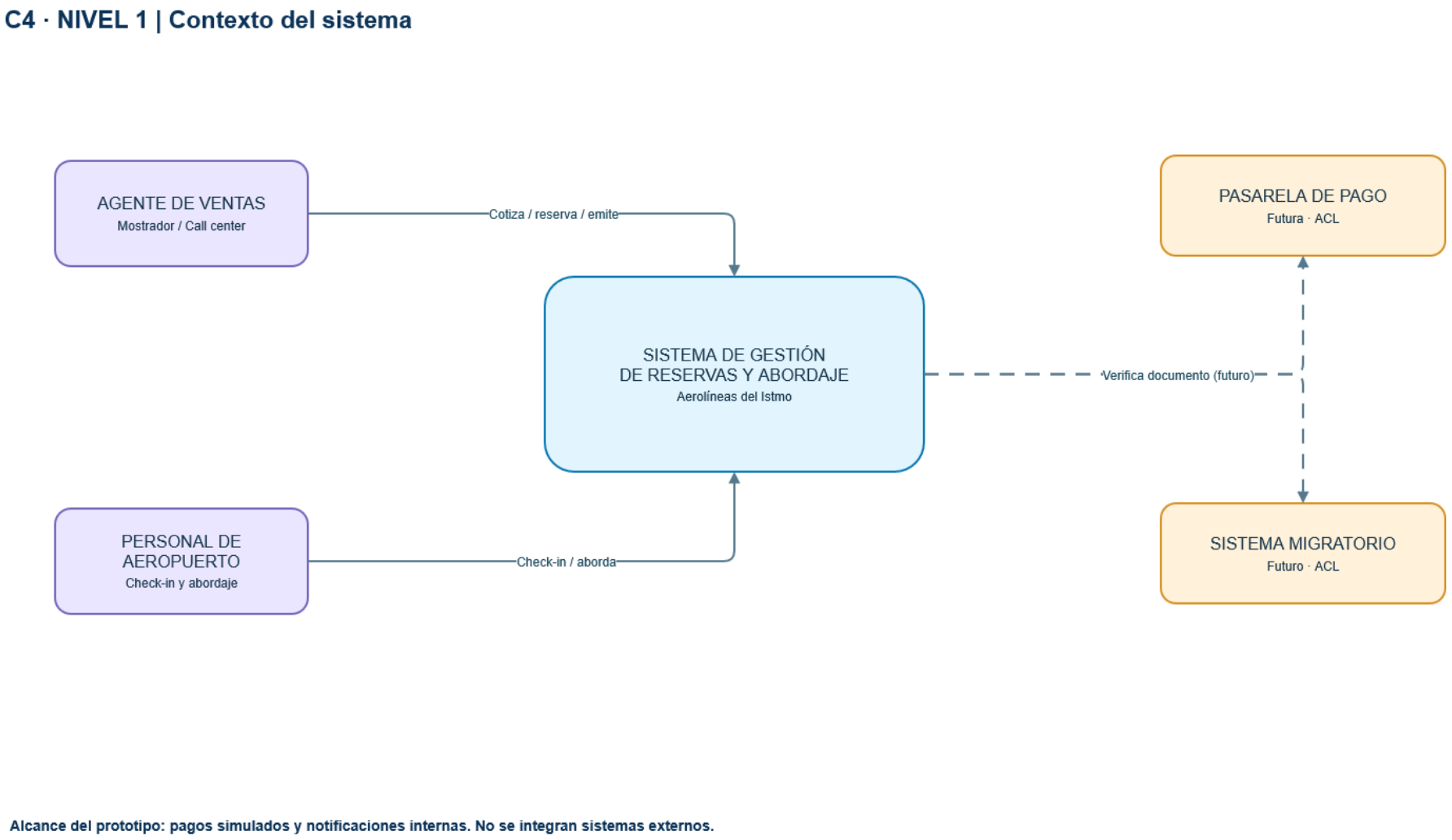
Diagrama 1 de 7 · Imagen proporcionada para la documentación del proyecto



Relaciones Cliente/Proveedor, Conformista, Kernel Compartido y capas anticorrupción. Los sistemas externos son integraciones futuras.

C4 · NIVEL 1 | CONTEXTO DEL SISTEMA

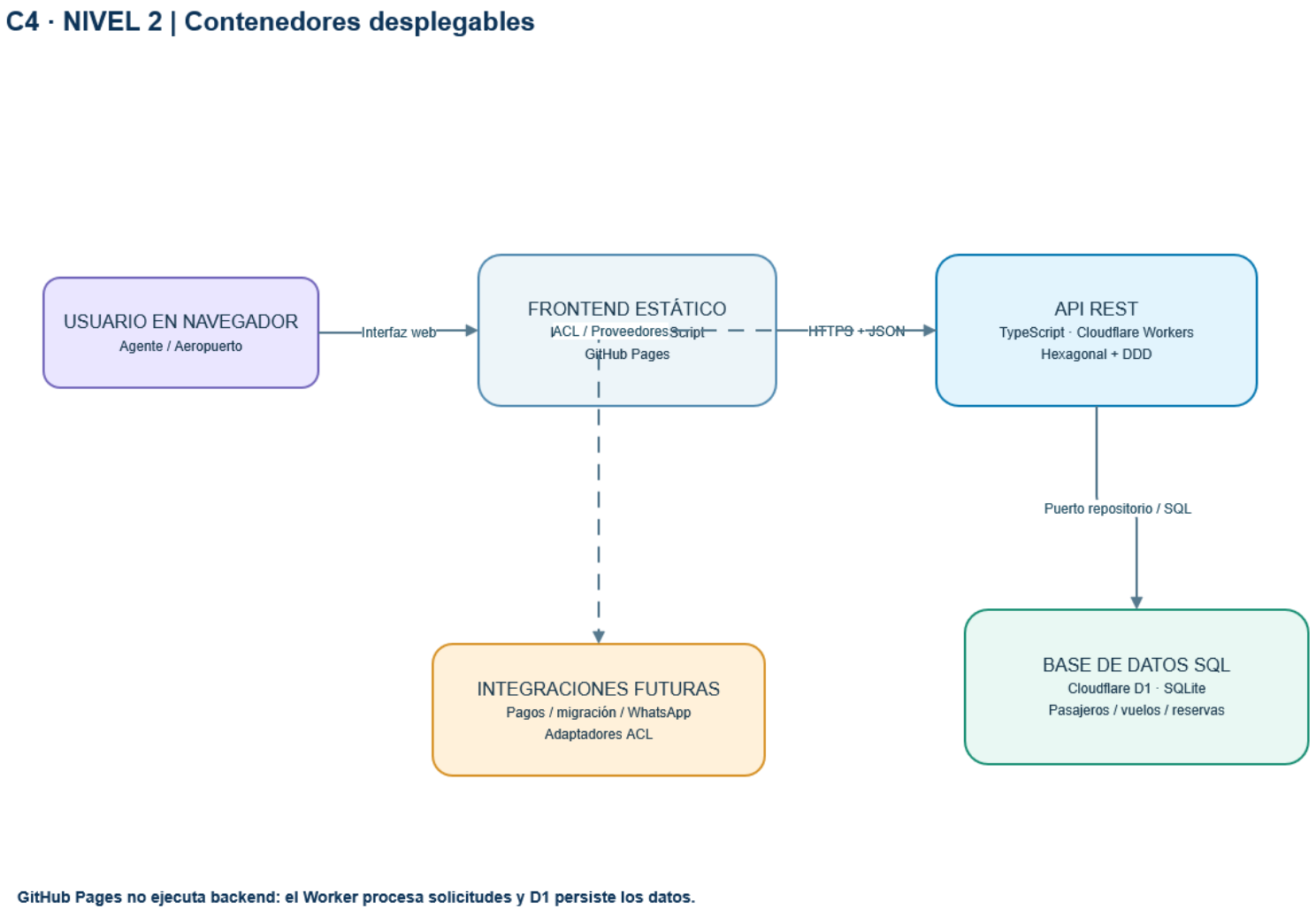
Diagrama 2 de 7 · Imagen proporcionada para la documentación del proyecto



Actores de ventas y aeropuerto, sistema principal y fronteras previstas con plataformas externas.

C4 · NIVEL 2 | CONTENEDORES

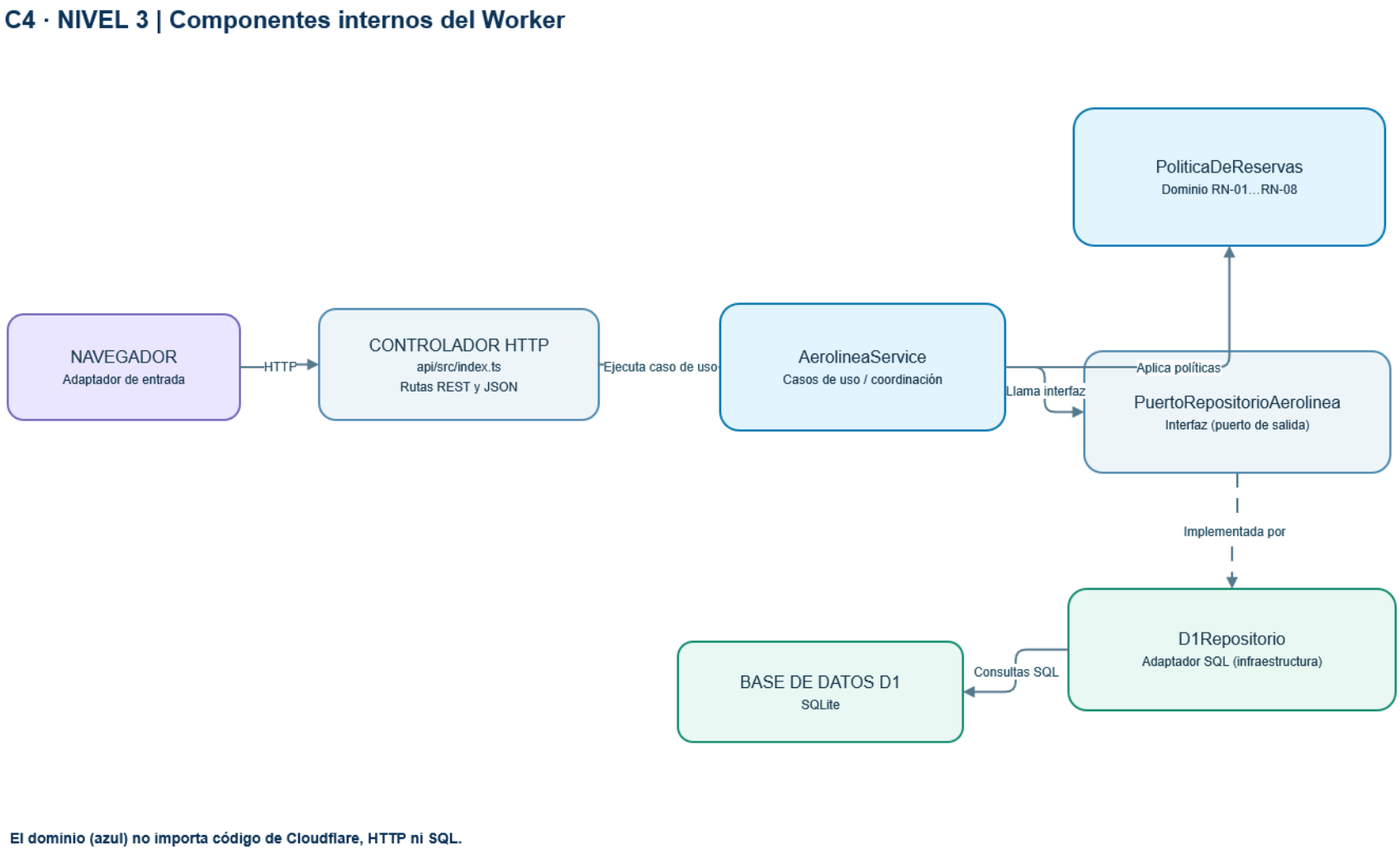
Diagrama 3 de 7 · Imagen proporcionada para la documentación del proyecto



El navegador utiliza GitHub Pages, que consulta por HTTPS la API TypeScript de Cloudflare Workers y su base D1.

C4 · NIVEL 3 | COMPONENTES

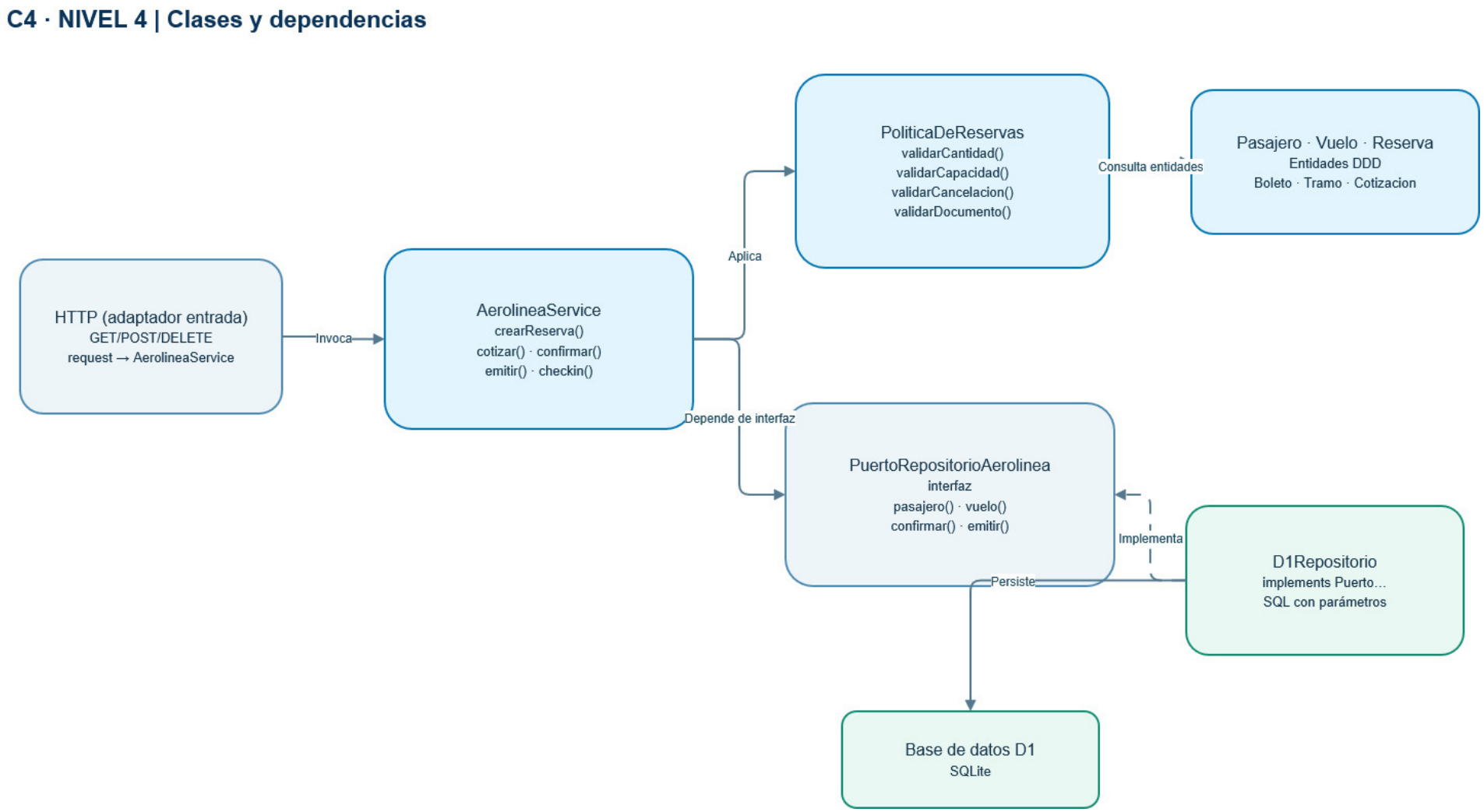
Diagrama 4 de 7 · Imagen proporcionada para la documentación del proyecto



El controlador HTTP llama a los casos de uso; las políticas del dominio se aíslan de SQL mediante un puerto de repositorio.

C4 · NIVEL 4 | CLASES Y DEPENDENCIAS

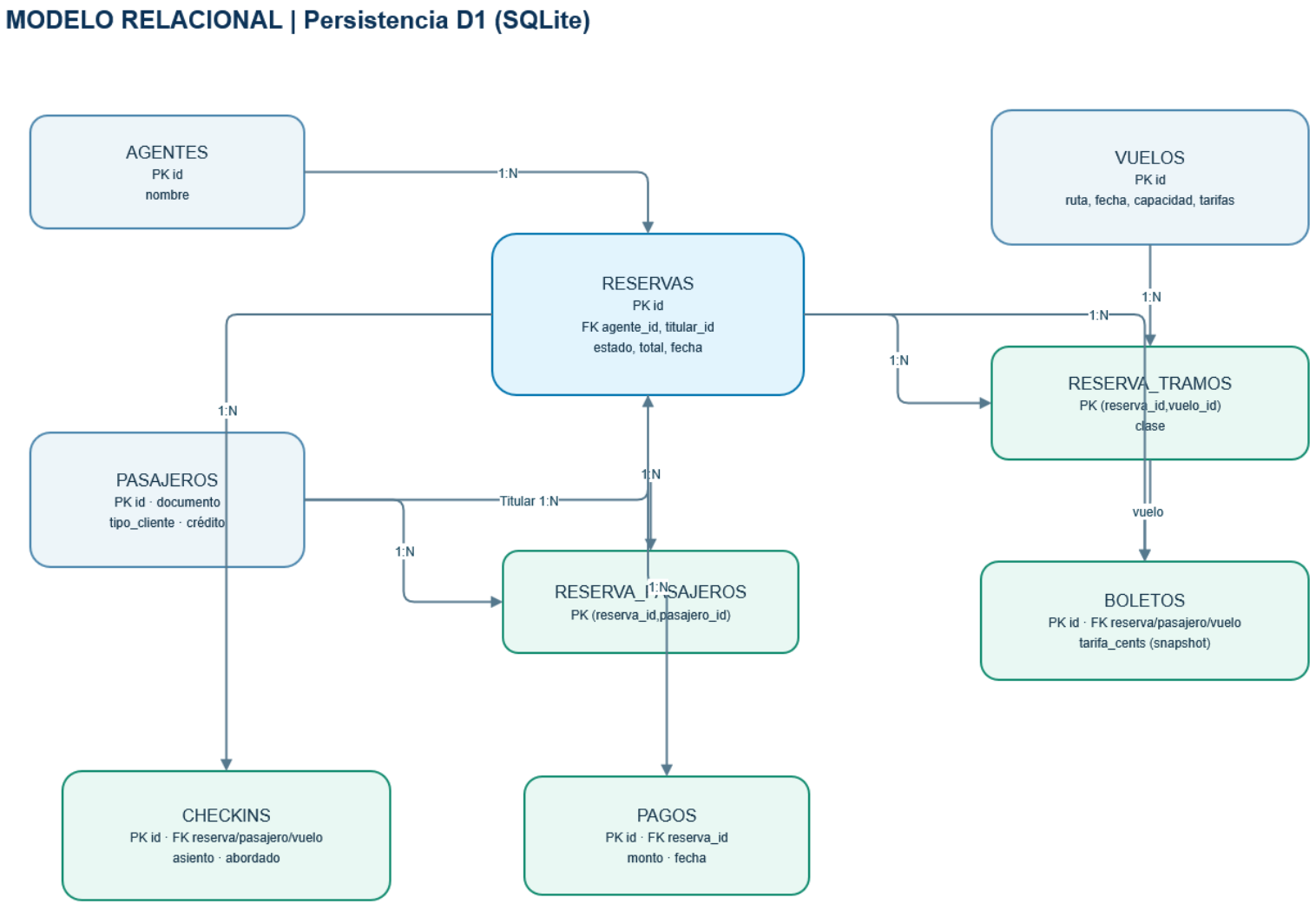
Diagrama 5 de 7 · Imagen proporcionada para la documentación del proyecto



Clases principales: AerolineaService, PoliticaDeReservas, PuertoRepositorioAerolinea y D1Repositorio.

MODELO RELACIONAL | D1 (SQLITE)

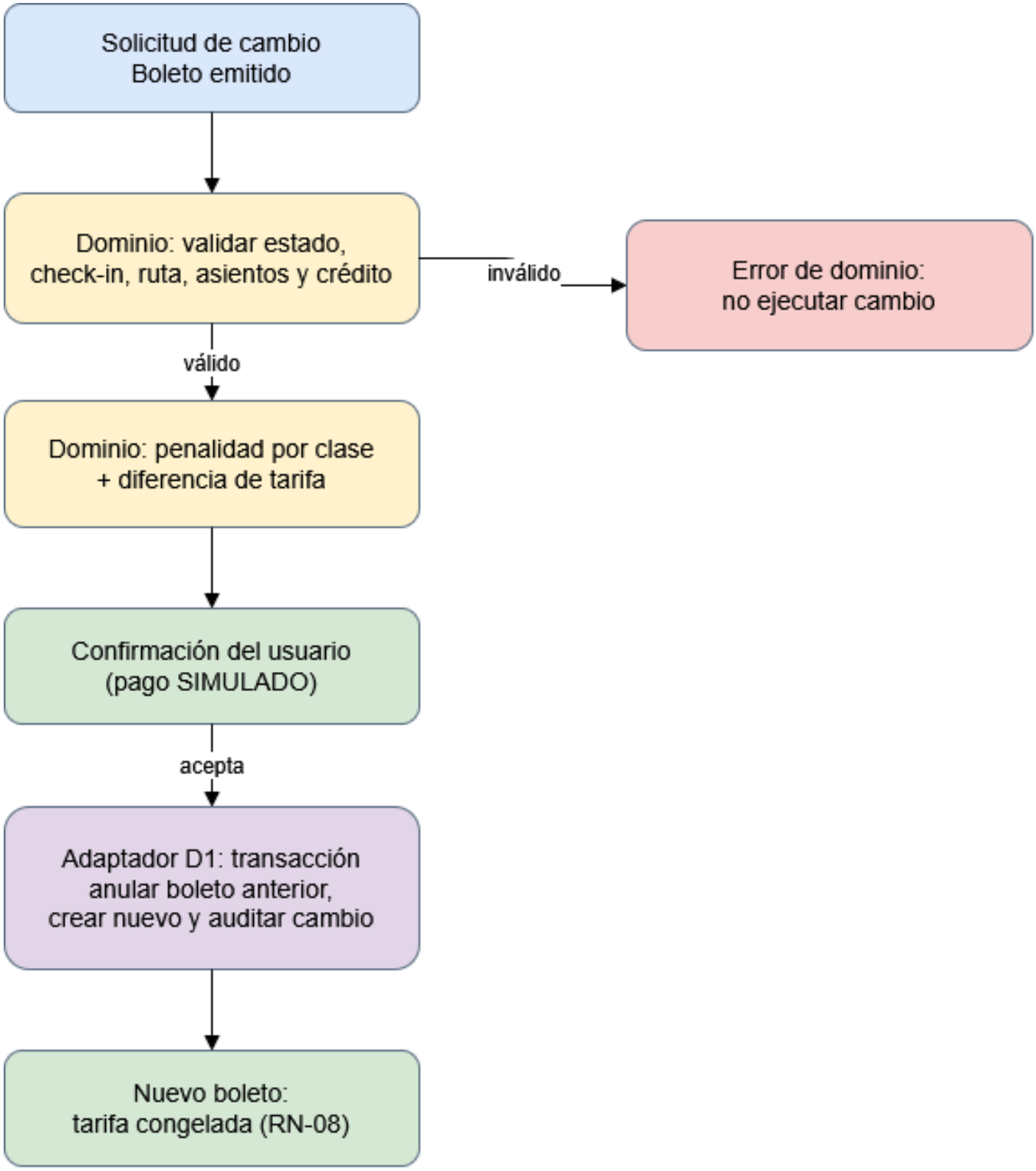
Diagrama 6 de 7 · Imagen proporcionada para la documentación del proyecto



Estructura de agentes, pasajeros, vuelos, reservas, tramos, boletos, pagos y check-ins; las migraciones amplían el esquema.

FLUJO DE CAMBIO DE BOLETO · RN-07

Diagrama 7 de 7 · Imagen proporcionada para la documentación del proyecto



La validación y el cálculo de penalidades pertenecen al dominio; el adaptador D1 persiste el cambio y se conserva la tarifa emitida.